MACHINE LEARNING MINIPROJECT

CODE:

app.py:

```
import streamlit as st
import numpy as np
import joblib
model = joblib.load("eye_strain_model.pkl")
st.title("👁️ VisionGuard: Eye Strain Risk Predictor")
st.write("Predict your eye strain risk based on screen usage habits.")
# Inputs
screen_time = st.slider("Screen time per day (hours)", 1, 14, 6)
brightness = st.slider("Screen brightness (%)", 10, 100, 60)
breaks = st.slider("Breaks per hour", 0.0, 4.0, 1.0, step=0.25)
font_size = st.slider("Font size (px)", 10, 32, 14)
blue_filter = st.selectbox("Blue light filter ON?", ("No", "Yes"))
blue_filter = 1 if blue_filter == "Yes" else 0
# Predict button
if st.button("Predict Eye Strain Risk"):
    input_data = np.array([[screen_time, brightness, breaks, font_size, blue_filter]])
    prediction = model.predict(input_data)[0]
    st.subheader(f"Predicted Eye Strain Level: **{prediction}**")
    # Recommendations
    st.write("---")
    st.subheader("Suggestions")
    if prediction == "High":
        st.write("""
        - Reduce screen time
        - Use blue light filter
        - Increase font size
        - Follow 20-20-20 rule
```

```
        - Take frequent breaks
        """)
    elif prediction == "Medium":
        st.write("""
        - Take regular breaks
        - Adjust brightness to comfortable level
        - Consider slightly larger font size
        """)
    else:
        st.write("✅ Good habits, keep maintaining a healthy screen routine!")
```

train_model.py:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.model_selection import GridSearchCV
import seaborn as sns
import matplotlib.pyplot as plt
import joblib
# Load dataset
df = pd.read_csv("eye_strain_dataset.csv")
# Encode target
y = df["strain_level"]
X = df.drop("strain_level", axis=1)
# Stratified split for balanced class performance
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
```

```
)

# Pipeline
pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression(multi_class='ovr', class_weight='balanced', max_iter=300))
])

# Grid search to tune regularization strength
param_grid = {
    "clf__C": [0.1, 1, 5, 10]
}

grid = GridSearchCV(pipe, param_grid, cv=5, scoring='accuracy')
grid.fit(X_train, y_train)
model = grid.best_estimator_
joblib.dump(model, "eye_strain_model.pkl")

# Predictions
train_pred = model.predict(X_train)
test_pred = model.predict(X_test)

# Accuracy
print("Best Hyperparameters:", grid.best_params_)
print("Training Accuracy:", accuracy_score(y_train, train_pred))
print("Testing Accuracy:", accuracy_score(y_test, test_pred))

# Confusion matrix
cm = confusion_matrix(y_test, test_pred)
print("\nClassification Report:")
```

```
print(classification_report(y_test, test_pred))
sns.heatmap(cm, annot=True, cmap="Blues", fmt='d',
        xticklabels=["Low","Medium","High"],
        yticklabels=["Low","Medium","High"])
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("True")
plt.show()
```

SCRIPT

Student 1:
Our mini project is titled *"VisionGuard: Eye Strain Risk Prediction Using Logistic Regression."*. It aims to estimate whether a person has a low, medium, or high risk of eye strain based on their daily screen usage habits. We created our own dataset of 250 samples using realistic inputs like screen time in hours, screen brightness percentage, font size in pixels, number of breaks per hour, and whether a blue light filter is used or not. These parameters were chosen because they directly affect visual comfort. The dataset was divided into training and testing parts for model building, and preprocessing was done using StandardScaler to normalize all features before training.

Student 2:
We used the Logistic Regression algorithm for classification because it's simple, efficient, and provides excellent accuracy for this type of dataset. After training and testing, our model achieved an overall accuracy of 96%, which shows it performs highly reliably. The confusion matrix confirmed this by showing that most predictions correctly matched the actual categories, with only a few overlaps between the Medium and High classes. We also generated a classification report, where precision, recall, and F1-scores were all above 0.9 — proving that the model generalizes well. The implementation was done completely in Python using Pandas, Scikit-learn, and Matplotlib.

Student 3:
After training, we deployed our model using a Streamlit dashboard for interactive real-time prediction. The user just enters their details like screen time, brightness, font size, breaks, and filter usage, and instantly gets a result showing whether the risk is Low, Medium, or High. Along with that, our app provides personalized eye-care recommendations to reduce strain. This makes the system both practical and user-friendly. In conclusion, our project demonstrates that even a simple algorithm like Logistic Regression can achieve high accuracy and serve as an effective health-assistance tool. In the future, we plan to expand it with live webcam-based eye tracking and deep learning models for more precise prediction.